

Minimal recorder: RAM consumption

Track N Race Minimal Recorder · Win32 build · measured 9 October 2026

Summary

- **It does not keep what it has written.** Over the whole race replayed at 16x, private memory stayed between 58 and 77 MB while it wrote a 76 MB recording, then fell to 21 MB when the file closed.
- **What it holds is the laps not yet written.** That means each of the 22 cars' current lap, plus finished laps waiting out the flashback window. A flashback can still rewind them, so they can't go to the file yet.
- **The only growth is the file's index,** about 10 MB over the race. The index is written when the file closes, so it has to stay in memory until then.
- **At real time it is the same.** A whole race at 1x stayed between 50 and 85 MB, ended at 84 MB and fell to 30 MB when the file closed (section 3).
- **Idle, it uses 7 MB.**

Windows 11, RTX 4050 laptop GPU, 1920 × 1080 at 125 %. Data: the 92-minute São Paulo race, as the UDP capture (udp_capture.bin, replayed with play_udp.py) or as its recording (udp_capture_sao_paulo_v6_2026-10-07.tnrd). **Private memory** is memory only this program uses; it is the number to compare. The **working set** is what currently sits in physical RAM, and Windows trims it freely.

1. A whole race

A fresh build of the recorder, recording to a scratch folder, with the 92-minute capture replayed at 16x (about 6 minutes). Memory read from Windows every 10 s. The recorder has no memory log of its own.

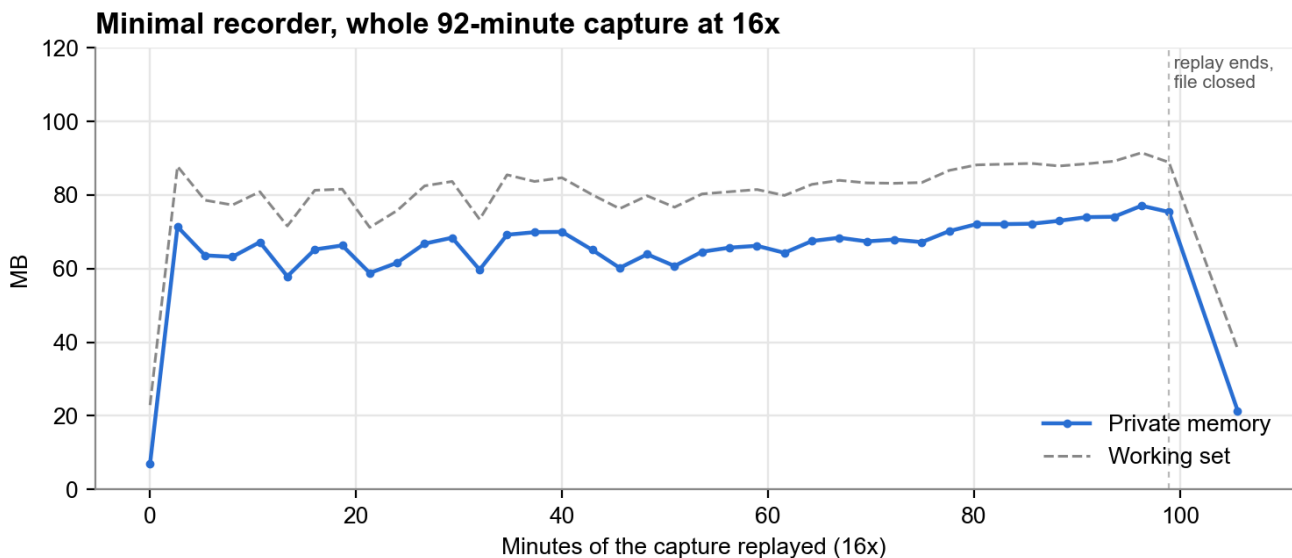


Figure 1. Private memory and working set against how much of the race has been replayed.

Point	Private	Working set
Idle, before any packets	7	23
Formation lap and lap 1	58–71	72–88
Mid-race	60–68	76–84
Last laps	72–77	88–92
After the session ended and the file closed	21	38

2. The first 15 minutes at real time

16x sends packets faster than a game does, so the same capture was also replayed at 1x for 15 minutes: the formation lap and about the first ten laps of the race.

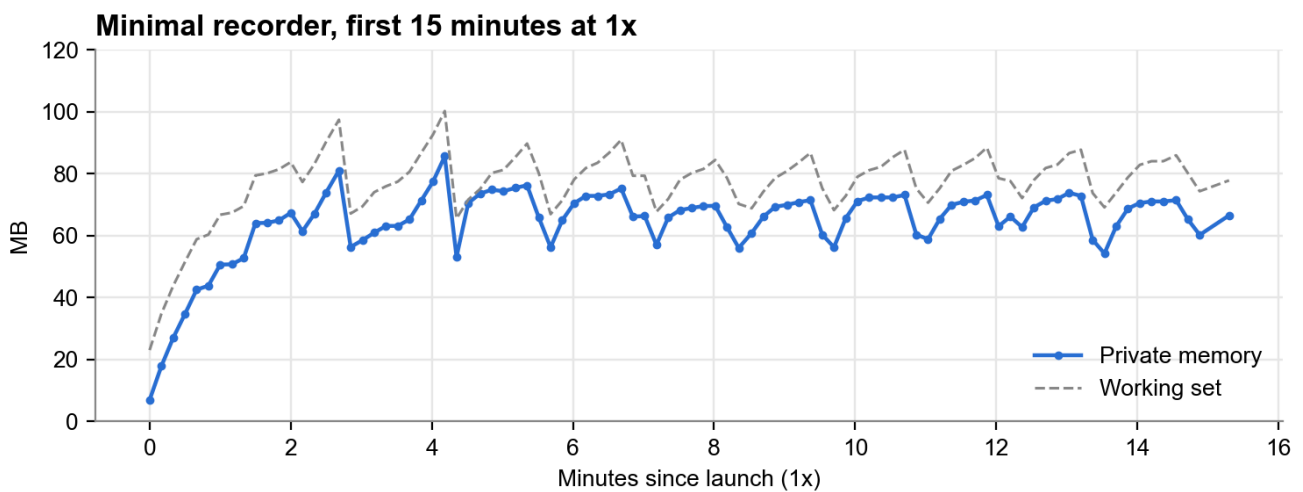


Figure 2. Private memory and working set at 1x, sampled every 10 s. The sawtooth has the period of a lap.

After the first 90 seconds, private memory stayed between 53 and 86 MB (median 69 MB), the same band as at 16x. It climbs through each lap and then drops by 10–30 MB about every 70–80 s, which is one lap at this track. That fits each finished lap being written to the file and its buffers released (inferred from the timing; the recorder has no log to confirm it).

3. A whole race at real time

On 9 October the recorder ran the whole capture at 1x, fresh from launch: 99 minutes, the formation lap and 71 laps, recording to a scratch folder. It wrote the same 76 MB recording. Memory read from Windows every 5 s.

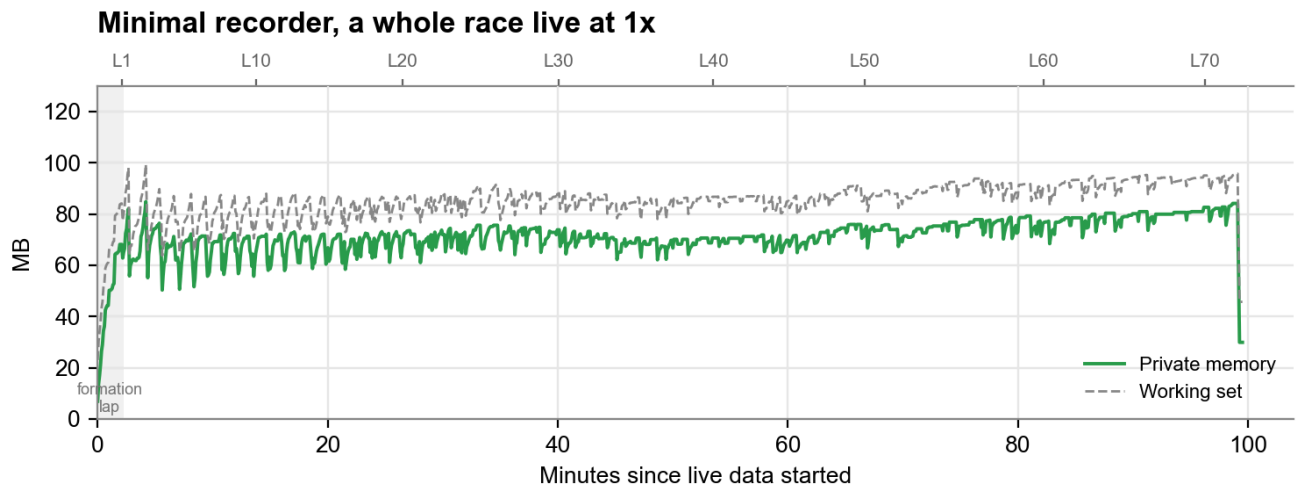


Figure 3. Private memory and working set over the whole race at 1x. Lap numbers run along the top. The drop at the end is the file closing.

Point	Private	Working set
Idle, before any packets	7	23
Minute 2	65	81
Minute 30	72	85
Minute 60	69	84
Minute 90	80	94
End of the race	84	94
After the session ended and the file closed	30	46

From minute 2 to the end of the race, private memory stayed between 50 and 85 MB, with the same once-a-lap sawtooth as the 15-minute run. The band drifts up by about 12 MB an hour (a straight-line fit from minute 35), close to the ~10 MB of index growth seen at 16x. The highest sample, 85 MB, came in minute 4, during the opening laps, not at the end. After the file closed the recorder settled at 30 MB, 9 MB above the 16x run's 21 MB (not investigated).

4. What stays in memory, and why

Held	Size	Why it can't go sooner
Each car's lap being built (22 cars, every data type)	Most of the ~60 MB	The lap isn't finished
Finished laps inside the write delay	Part of the ~60 MB	A flashback can still rewind them
Chunk index: one ~60-byte entry per written chunk, lap summaries	Grows ~10 MB a race	The index is written when the file closes
Last shared record of each type (session, participants)	Kilobytes	The next one is compressed against it
Strategy's state and rollback journal	Up to ~31 MB	Runs inside the shared engine
Compression context and scratch buffer	A few MB	Reused for every chunk

The full-session memory copy is off. It is controlled by `Config::binaryPlayback`, which feeds Electron's charts and which the recorder doesn't set. Each finished chunk is compressed and appended straight to the file, and only its index entry is kept (`appendEncoded` in `TNRD_V6.cpp`). `MinimalController` itself holds nothing: it watches session rows and the recording folder.

5. What's proven and what isn't

Claim	Status
Memory stays flat over a whole race while the file grows to 76 MB	Measured (16x)
The same band at 1x	Measured over the whole race (section 3)
The once-a-lap drops are laps being written and released	Inferred from their timing
Written chunks are not kept in memory	Confirmed in code and consistent with the measurement
The ~10 MB of growth is the index	Inferred from the code; not measured item by item
A whole race at 1x: 50–85 MB, 84 MB at the end, 30 MB after the file closed	Measured (9 October, one run)
Why it settles at 30 MB after a 1x race and 21 MB after a 16x one	Not investigated

6. Next steps for optimization

Ordered by expected saving for the effort. None of them drops or thins out data. **Saving** says whether the figure is measured, computed or an estimate.

#	Step	Where	Saving	Effort and risk
1	Switch Strategy off in the recorder	Engine: <code>Config</code> , <code>Engine.cpp</code>	Up to ~31 MB	Low. Recordings are unaffected
2	Size the writer's lap builders to what they hold	Engine: V6 writer builders	Up to ~10 MB (estimate)	Low
3	Write laps sooner	Engine: V6 writer write delay	Part of the ~60 MB floor	High. Breaks flashback handling; not recommended

Notes

- **Strategy (1).** The engine feeds Strategy on every live row whatever is subscribed (`Engine.cpp`, around line 1138). Strategy's results are never written to the recording: the writer only receives the parser's rows (`Engine.cpp`, line 1080). So the recorder could turn Strategy off and lose nothing from its files. That needs a switch in `tnrp::Config`, defaulting to on for the other apps.
- **Lap builders (2).** In the Qt run the writer's builders held 20.3 MB of capacity for 8.9 MB of pending laps. The recorder uses the same writer. Sizing each new lap's builders from the previous lap's row counts would avoid most of the growth slack.
- **Write delay (3).** Finished laps wait out the flashback window before they're written. Shortening it would lower the ~60 MB floor, but a flashback past a written lap would then be recorded wrongly. Not recommended.

7. Observations

Overall: good. Nothing to fix. It behaves the way a recorder should: memory set by the laps in flight, not by the length of the race.

- **Flat is what matters, and it is flat.** Memory rises and falls with each lap instead of accumulating; the only drift is the index, about 12 MB an hour at 1x, so a two-hour session would end about 20–25 MB above a short one.
- **The ~60 MB floor is the price of handling flashbacks.** Writing laps sooner would shrink it, but a flashback past a written lap could then no longer be recorded correctly. I'd keep it.
- **Strategy runs even though nothing displays it.** It is a candidate to switch off in this app if the last ~30 MB matter. That would need a configuration switch in the shared engine.
- **For scale:** over the same whole race at 1x, the Qt app's engine went from 71 to 150 MB and Electron's main process from 166 to 306 MB, both with the in-memory copy of the recording on. The recorder is the engine with little else around it.